Sanaverkko Algorithms and Background

Updated Feature Documentation

Project Documentation

March 10, 2026

Contents

1 Overview **2**

2 New Features and How to Use Them **3**
- 2.1 Feature summary . . . 3
- 2.2 Quick usage workflow . . . 4

3 Mathematical Formulation **5**
- 3.1 Gematria and number features . . . 5
- 3.2 Reference-candidate base distance . . . 5
- 3.3 LTM blending and common-word penalty . . . 5
- 3.4 Phoneme rhyme bonus . . . 6
- 3.5 Final blended ranking score . . . 6
- 3.6 Activation and backprop style weight update . . . 6

4 Pseudocode Examples **7**
- 4.1 Candidate retrieval + scoring + selection . . . 7
- 4.2 Sentence logic iteration . . . 8
- 4.3 Main simulation step . . . 8
- 4.4 LTM training CLI example . . . 9

5 Practical Configuration Recipes **10**
- 5.1 Stable symbolic mode . . . 10
- 5.2 Balanced creative mode . . . 10
- 5.3 Chaotic exploration mode . . . 10

6 Notes and Limitations **11**

Source Basis **12**

Chapter 1

Overview

Sanaverkko is an experimental word-network system that combines:

- gematria-driven symbolic transformations,
- optional POS-aware linguistic constraints,
- optional long-term memory (LTM) candidate re-ranking,
- real-time neural-like activation dynamics,
- and real-time audio synthesis from sentence state.

The application is centered in `sanaVerkkoCore.py` with audio in `sanasyna.py` and optional LTM training/inference support in `sv_ltm.py`.

Chapter 2

New Features and How to Use Them

This chapter lists the practical feature set currently present in the project and how to use each feature from the UI.

2.1 Feature summary

Feature	What it does	How to use
Fluid Gematria	Expands candidate search from strict exact gematria to reduction and digital-root buckets.	Enable `Fluid gematria` checkbox. Keep OFF for strict mode.
Fluid Root (jump control)	Allows jump-search across different digital roots when jump mode is used.	Enable `Fluid root` for freer jumps; disable for root-preserving jumps.
Jump Probability + Radius	Escapes local minima by sampling near gematria buckets around source value.	Increase `Jump probability` and `Jump radius (gematria)` gradually.
Selection Exploration + Top-k	Mixes deterministic best-candidate choice with weighted stochastic exploration.	Tune `Selection exploration (0-1)` and `Selection top-k`.
LTM Candidate Re-ranking	Uses `.svltm` next-word probabilities to bias candidate ranking.	Load model with `Load long term memory`; enable `Use long term memory`; tune `LTM weight`.
Common Word Penalty	Penalizes high-frequency filler words in LTM scoring.	Toggle `Common word penalty`. Keep ON for more varied lexical output.
Phoneme Rhyme Scoring	Adds rhyme bonus when source and candidate rhyme tails are similar.	Keep `Use phoneme rhyme` ON; tune `Rhyme weight` and `Rhyme min similarity`.
POS Matching + Fluid POS	Constrains substitutions by part-of-speech, optionally fixed to seed POS or dynamic POS.	Enable `Use POS matching`; enable `Fluid POS` for adaptive POS.
Indexed Candidate Retrieval	Fast lookups by gematria/reduction/root and optional POS-crossed indexes.	Automatic after importing/adding words; no manual action required.

Feature	What it does	How to use
Preset Save/Load JSON	Stores and restores nearly all simulation/audio/selection parameters.	Use `Save preset` and `Load preset`.
Import Mode Append/Replace	Controls whether imported text extends or replaces reference DB.	Select mode from `Import mode` before `Import .txt`.
Polyphony + Voice Spread	Multi-voice melody rendering with detuned spread for richer sound.	Set `Polyphony voices (1-4)` and `Voice spread`.
Frequency Mapping Modes	Maps generated frequencies to original, Pythagorean, or equal-tempered modal families.	Select `Frequency mapping`.
Waveform Modes	Switches synthesis character among dynamic, pure sine, noise-heavy, classic analog.	Select `Audio waveform mode`.
ADSR + Envelope Display	Controls attack/decay/sustain/release and visualizes envelope shape in control panel.	Edit ADSR fields and observe envelope panel.
Minimum Note Duration + Melody Speed	Prevents too-short notes and globally scales temporal pacing.	Tune `Minimum note duration (s)` and `Melody speed coeff`.
Live Output Window	Shows ongoing `output.txt` updates in separate window.	Opens automatically with app; monitor generated sentence traces.
Explicit Numeric Commit	Numeric text fields apply values on Enter and revert invalid input.	Type value and press Enter in each field.

2.2 Quick usage workflow

1. Start app: `python sanaVerkkoCore.py`
2. Add words or import database text.
3. Choose strict mode first (Fluid Gematria OFF), then widen exploration gradually.
4. Optionally load LTM and enable it.
5. Tune exploration/jump/rhyme/POS parameters for desired sentence motion.
6. Tune audio mapping/waveform/polyphony/ADSR.
7. Save a preset once behavior is stable.

Chapter 3

Mathematical Formulation

3.1 Gematria and number features

For a word $w = c_1 c_2 \dots c_n$ and character-value map $v(\cdot)$:

$$G(w) = \sum_{i=1}^{n} v(c_i). \tag{3.1}$$

One-pass numerological reduction:

$$R(x) = \sum_{d \in \mathrm{digits}(x)} d. \tag{3.2}$$

Digital root:

$$D(x) = \underbrace{R(R(\dots R(x)\dots))}_{\text{until } x<10}. \tag{3.3}$$

3.2 Reference-candidate base distance

Given source word s and candidate c:

$$\Delta_G = \frac{|G(c) - G(s)|}{1000}, \tag{3.4}$$

$$\Delta_R = \frac{|R(G(c)) - R(G(s))|}{9}, \tag{3.5}$$

$$\Delta_D = \frac{|D(G(c)) - D(G(s))|}{9}. \tag{3.6}$$

Base score (smaller is better):

$$\mathrm{base}(s, c) = \Delta_G + 0.35\Delta_R + 0.35\Delta_D. \tag{3.7}$$

3.3 LTM blending and common-word penalty

Let raw model probability be $p_{\mathrm{raw}}(c \mid \text{context})$, and penalty factor $\pi(c)$:

$$\pi(c) = \begin{cases} 0.35, & c \text{ in common-word list and penalty enabled,} \\ 1.0, & \text{otherwise.} \end{cases} \tag{3.8}$$

Adjusted probability and LTM term:

$$p_{\mathrm{adj}} = p_{\mathrm{raw}} \cdot \pi(c), \tag{3.9}$$

$$\mathrm{ltmTerm} = 1 - p_{\mathrm{adj}}. \tag{3.10}$$

3.4 Phoneme rhyme bonus

A grapheme-to-phoneme approximation computes phoneme tails from the last vowel onward. Let $\rho(s, c) \in [0, 1]$ be suffix-tail similarity. If $\rho(s, c)$ exceeds threshold τ, then

$$\text{rhymeBonus}(s, c) = \rho(s, c), \tag{3.11}$$

else it is 0.

3.5 Final blended ranking score

With LTM weight $\lambda \in [0, 1]$ and rhyme weight $\omega \in [0, 1]$:

$$\text{score}(s, c) = (1 - \lambda)\,\text{base}(s, c) + \lambda\,\text{ltmTerm} - \omega\,\text{rhymeBonus}(s, c). \tag{3.12}$$

Final score is clamped to nonnegative values:

$$\text{score}(s, c) \leftarrow \max(0, \text{score}(s, c)). \tag{3.13}$$

3.6 Activation and backprop style weight update

Neuron activation update (code-level form):

$$a_i \leftarrow a_i + \text{sign}(a_i)\,\alpha + \sum_{j \in \mathcal{N}(i)} \sigma(a_j; s)\, w_{ij}\, u, \tag{3.14}$$

where α is `activation_increase`, s is `sigmoid_scale`, and u is external step input.

Connection weight learning:

$$e_i = (t - a_i)\,\gamma, \tag{3.15}$$

$$\Delta w_{ij} = \eta\, e_i\, \sigma(a_j; s), \tag{3.16}$$

$$w_{ij} \leftarrow \text{clip}(w_{ij} + \Delta w_{ij}, -W_{\max}, W_{\max}), \tag{3.17}$$

with η = learning rate, γ = error gain, and $W_{\max} = \max(0.1, 4 \cdot \texttt{activation_limit})$.

Chapter 4

Pseudocode Examples

4.1 Candidate retrieval + scoring + selection

```
function FIND_BEST_REFERENCE(sourceWord):
    usePOS <- is_pos_matching_enabled()
    fluidGematria <- is_fluid_gematria_enabled()
    fluidRoot <- is_fluid_root_enabled()

    ensure_reference_index(includePOS = usePOS)
    candidates <- {}

    if usePOS:
        sourcePOS <- (fluid POS ? current POS : seed POS)
        add index.pos_gematria[(sourcePOS, G(sourceWord))]
        if fluidGematria:
            add index.pos_reduction[(sourcePOS, R(G(sourceWord)))]
            add index.pos_root[(sourcePOS, D(G(sourceWord)))]

    if candidates empty:
        add index.gematria[G(sourceWord)]
        if fluidGematria:
            add index.reduction[R(G(sourceWord))]
            add index.root[D(G(sourceWord))]

    shouldJump <- random() < jump_probability
    if shouldJump and fluidGematria:
        keys <- nearby gematria buckets within jump_radius
        if not fluidRoot: keep only keys with same digital root
        add candidates from selected gematria buckets

    if not fluidGematria:
        shouldJump <- false
        keep only exact gematria candidates

    return SELECT_BEST(sourceWord, candidates, forceJump = shouldJump)

function SELECT_BEST(sourceWord, candidates, forceJump):
    ltmProbs <- LTM candidate probabilities (optional)
    ranked <- sort candidates by tuple:
              (blendedScore, baseScore, gematriaDelta,
               reductionDelta, rootDelta, candidateWord)

    if forceJump:
        pick random from non-self candidates in widened ranked slice
```

```
    if random() < selection_exploration:
        pick weighted random from top-k (weight = 1/(rankIndex+1))

    if best candidate is source itself and alternatives exist:
        return first alternative
    return best candidate
```

4.2 Sentence logic iteration

```
function ITERATE_SENTENCE_TO_LOGIC(maxIterations):
    seenStates <- empty set
    changedAny <- false

    repeat up to maxIterations:
        state <- tuple(current sentence words)
        if state in seenStates: break
        add state to seenStates

        changedThisRound <- false
        for each word in sentence:
            if abs(word.activation) > word_change_threshold:
                if changeWord(word):
                    changedThisRound <- true

        if not changedThisRound: break
        changedAny <- true

    return changedAny
```

4.3 Main simulation step

```
function SIMULATION_STEP():
    if not enough time since last step: return

    process UI/window events
    logicTriggered <- false

    for each word in network:
        word.activate(sin(current_time))
        word.neuron.backpropagate(target)

        if abs(word.activation) > 2:
            word.activation <- 1

        if abs(word.activation) > word_change_threshold:
            logicTriggered <- true

    if logicTriggered:
        sentenceChanged <- ITERATE_SENTENCE_TO_LOGIC(logic_iteration_limit)

    update audio from current sentence/activations
    draw network and labels
```

```
    if sentenceChanged:
        write sentence, gematria expression, and reduction chain to output.txt
```

4.4 LTM training CLI example

```
# Train .svltm model
python sv_ltm.py --input kalevala.txt --output kalevala_balanced.svltm \
  --epochs 20 --context 3 --hidden1 1024 --hidden2 512 --device auto

# In app:
# 1) Load long term memory -> choose .svltm
# 2) Enable Use long term memory
# 3) Tune LTM weight and Common word penalty
```

Chapter 5

Practical Configuration Recipes

5.1 Stable symbolic mode

- Fluid Gematria OFF, Fluid Root OFF.
- Low exploration (≤ 0.1), low jump probability (≤ 0.05).
- POS matching ON, Fluid POS OFF.

5.2 Balanced creative mode

- Fluid Gematria ON, Fluid Root OFF.
- Exploration around 0.15–0.25, top-k around 4–6.
- Moderate rhyme weight and LTM weight ($\approx$ 0.3–0.5).

5.3 Chaotic exploration mode

- Fluid Gematria ON, Fluid Root ON.
- Jump probability ≥ 0.2, larger jump radius.
- Higher exploration and larger top-k.

Chapter 6

Notes and Limitations

- Gematria/numerology are used as symbolic transformation signals, not empirical linguistic truth metrics.
- POS fallback heuristics are intentionally lightweight when NLTK resources are unavailable.
- LTM guidance is optional and corpus-dependent.
- Output quality strongly depends on reference database quality and parameter settings.

Source Basis

This document is based on current repository implementation and docs, primarily:

- `sanaVerkkoCore.py`
- `sanasyna.py`
- `sv_ltm.py`
- `README.md`
- `LTM_USAGE.md`